

Programação Orientada a Objetos

Palavras chaves static, final e Classes Abstratas

Professor: Me. Sergio Souza Novak
Faculdade de Engenharia de Software



1 Modificadores **static** e **final**

- O Modificador **static**
- O Modificador **final**

2 Fundamentos de Classes Abstratas

- Abstração e Sintaxe
- Métodos Abstratos

3 Estrutura e Implementação

- UML e Implementação
- Construtores e `super()`

4 Polimorfismo e Casting

- Tipos de Referência
- Casting e Conversão

5 SOLID e Boas Práticas

- Inversão de Dependência
- Modificadores e Regras de Design

Modificadores static e final

1. Modificadores de Acesso e Escopo

- Antes de avançarmos para abstração, precisamos entender dois modificadores cruciais no Java: `static` e `final`.
- Eles controlam como os membros de uma classe são armazenados na memória e se podem ser alterados ou estendidos.

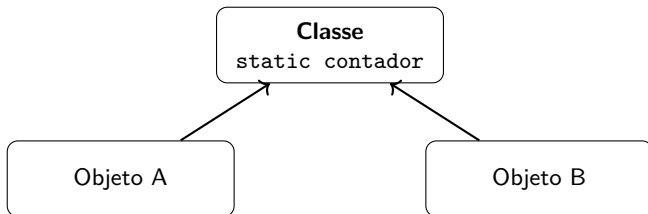
Destaque

Enquanto `public/private` controlam a **visibilidade**, `static/final` controlam o **comportamento** e a **existência**.



2. O Modificador `static`

- Pertence à **classe**, não ao objeto.
- **Único na memória**: criado uma vez ao carregar a classe.
- **Compartilhado**: todos os objetos usam o mesmo valor.
- **Acesso**: `Classe.membro`



Ideia-chave

Existe apenas **uma cópia** do membro `static`, compartilhada por todos os objetos.

3. Atributos static

- Úteis quando todos os objetos precisam compartilhar a mesma informação.
- **Exemplos comuns:**
 - Contador de objetos criados.
 - Taxa de juros padrão de um banco.
 - Configurações globais do sistema.



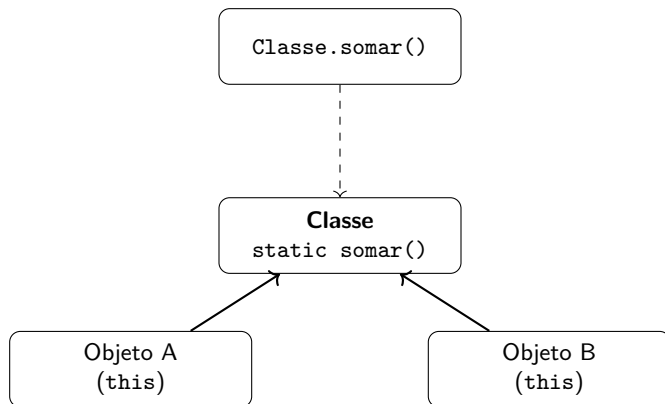
4. Exemplo: Atributo static

```
1 class Carro {
2     static int totalCarros = 0; // Compartilhado
3     String modelo;
4
5     Carro(String modelo) {
6         this.modelo = modelo;
7         totalCarros++; // Incrementa para todos
8     }
9 }
10
11 // Uso:
12 System.out.println(Carro.totalCarros);
```

Listing: Contador Global

5. Métodos static

- Não dependem do estado de um objeto.
- Pertencem à **classe**, não à instância.
- **Restrição:** não acessam membros não-estáticos diretamente (não existe `this`).
- Ideais para **métodos utilitários** (ex: cálculos).



6. Exemplo: Métodos static

```
1 class Calculadora {  
2     // Metodo static: nao precisa de objeto  
3     public static int somar(int a, int b) {  
4         return a + b;  
5     }  
6 }
```

Formas de Invocação

```
1 // Forma recomendada (pela classe)  
2 int total1 = Calculadora.somar(10, 20);  
3  
4 // Funciona, mas NAO recomendado  
5 Calculadora calc = new Calculadora();  
6 int total2 = calc.somar(5, 3);
```

Observação

Métodos static devem ser chamados pela classe, pois não dependem de objetos.

10. Resumo: Membros de Classe vs Instância

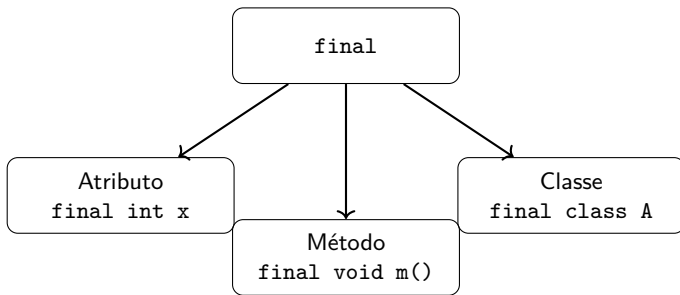
Característica	Sem Static	Com Static
Pertence a	Ao objeto (instância)	À classe
Cópia na Memória	Uma por objeto	Única para todos
Acesso via	Referência (obj.)	Nome da classe (Classe.)
Uso do this	Permitido	Proibido

7. O Modificador final

- Indica algo **imutável** ou que não pode ser modificado.

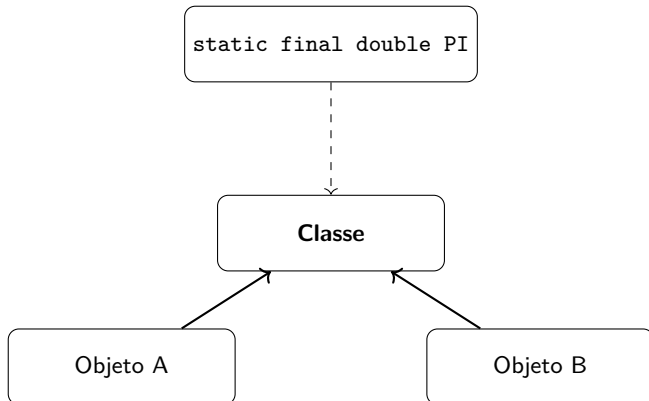
Aplicações do final

- **Atributo:** valor constante (não pode ser alterado)
- **Método:** não pode ser sobrescrito (`@Override`)
- **Classe:** não pode ser herdada (`extends`)



8. Atributos final (Constantes)

- Uma variável `final` só pode receber um valor **uma única vez**.
- Se for `static final`, temos uma **constante global**.
- Convenção: nomes em **MAIÚSCULO**.



8.1. Erro: Alterar Atributo final

- O compilador garante que uma variável `final` não seja modificada após a inicialização.
- Tentar mudar o valor gera um erro de compilação (*cannot assign a value to final variable*).

Exemplo de Erro

```
1 class Teste {  
2     final int LIMITE = 100;  
3  
4     void mudar() {  
5         LIMITE = 200; // ERRO: impossivel alterar!  
6     }  
7 }
```



8.2. Erro: Herança e Sobrescrita final

- Classes final bloqueiam a extensão (extends).
- Métodos final bloqueiam o polimorfismo (@Override).

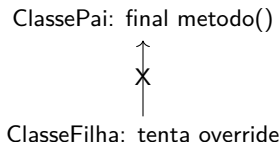
Exemplos de Bloqueio

```
1 // Erro na Classe
2 final class Seguranca { }
3 class Tentativa extends Seguranca { } // ERRO!
4
5 // Erro no Metodo
6 class Pai {
7     final void segredo() { }
8 }
9 class Filho extends Pai {
10     void segredo() { } // ERRO: impossivel sobrescrever!
11 }
```

9. Métodos e Classes final

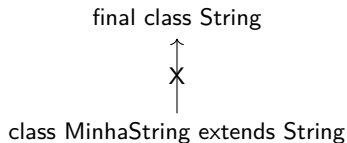
Método final

Garante que a lógica do método não pode ser sobrescrita por subclasses.



Classe final

A classe não pode ser herdada. **Exemplo:** String



Fundamentos de Classes Abstratas

1. Por que Classes Abstratas?

- Frequentemente, em modelagens OO, criamos classes muito genéricas (ex: `Animal`, `FormaGeometrica`).
- Faz sentido instanciar um objeto do tipo "`Animal`"?
- **Resposta:** Geralmente não. Instanciamos tipos específicos: `Cachorro`, `Gato`, `Circulo`.
- **Classes Abstratas** existem para modelar esses conceitos abstratos que servem apenas como base para outros.



2. Conceito Fundamental

- Uma classe abstrata é declarada com a palavra-chave `abstract`.
- **Não pode ser instanciada diretamente** (`new Animal()` gera erro de compilação).
- Serve exclusivamente como **superclasse** para outras classes.

Sintaxe Básica

```
abstract class Animal { ... }
```



3. Exemplo: Impossibilidade de Instanciação

```
1 abstract class Animal {  
2     abstract void emitirSom();  
3 }  
4  
5 public class Teste {  
6     public static void main(String[] args) {  
7         // ERRO DE COMPILACAO:  
8         // Animal animal = new Animal();  
9     }  
10 }
```

Listing: Tentativa de instanciar classe abstrata

4. Métodos Abstratos

- São métodos declarados com `abstract` que **não possuem implementação** (sem corpo `{}`).
- Definem **o que** deve ser feito, mas não **como**.
- O corpo do método é substituído por um ponto e vírgula `;`.

Assinatura

```
abstract void emitirSom();
```



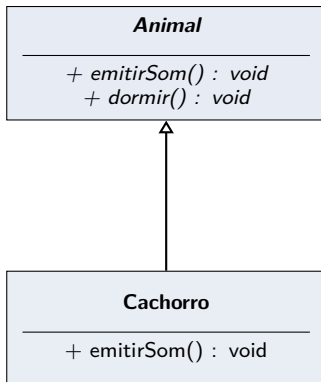
5. Classes Concretas e Obrigatoriedade

- Uma classe **concreta** (não abstrata) que herda de uma classe abstrata **deve obrigatoriamente** implementar todos os métodos abstratos da superclasse.
- Caso não implemente, ela também deve ser declarada como abstract.
- Isso garante que qualquer objeto instanciável tenha todos os métodos definidos.



Estrutura e Implementação

6. Diagrama de Classes (UML)



← Abstrata (itálico em UML)

7. Métodos Concretos (Com Implementação)

- Nem tudo em uma classe abstrata precisa ser abstrato.
- Elas podem conter métodos com corpo (implementados).
- **Vantagem:** Reaproveitamento de lógica comum a todas as subclasses.

Exemplo

Todos os animais "dormem" da mesma forma, mas "emitem som" de formas diferentes.



8. Código: Misturando Métodos

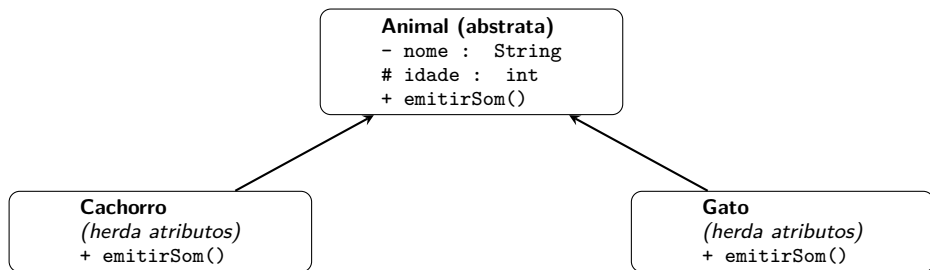
```
1 abstract class Animal {  
2     // Metodo Abstrato (sem corpo)  
3     abstract void emitirSom();  
4  
5     // Metodo Concreto (com corpo)  
6     void dormir() {  
7         System.out.println("Zzzzz...");  
8     }  
9 }
```

9. Atributos e Estado

- Classes abstratas podem possuir atributos normalmente (`private`, `protected`, `public`).
- Permitem encapsular o estado comum que será herdado.
- Exemplo: Todo animal possui um nome e idade.



10. Modificadores de Acesso e Herança



- - private: acessível apenas dentro de **Animal**
- # protected: acessível em **Cachorro** e **Gato**
- + public: acessível por qualquer classe

- Mesmo sendo projetadas para herança, classes abstratas também precisam **encapsular comportamentos internos**.
- Métodos `private` são utilizados para:
 - **Ocultar lógica interna** que não deve ser exposta às subclasses.
 - **Evitar sobrescrita** (*override*), garantindo comportamento fixo.
 - **Modularizar código**, dividindo algoritmos complexos em partes menores.
- Muito comum no padrão **Template Method**:
 - Um método `public` ou `final` define o fluxo geral.
 - Métodos `private` implementam etapas internas do algoritmo.
- Exemplo conceitual:
 - Classe abstrata define um processo geral.
 - Partes críticas do processo ficam em métodos `private`.
 - Subclasses apenas implementam os pontos de extensão necessários.
- Benefícios:
 - Maior **segurança** e controle do comportamento.
 - Melhor **manutenibilidade**.
 - Redução de erros por uso indevido pelas subclasses.

10. Construtores em Classes Abstratas

- **Dúvida Comum:** Se não posso instanciar, por que ter construtor?
- **Resposta:** O construtor é usado pelas subclasses via `super()`.
- Serve para inicializar os atributos definidos na classe abstrata de forma padronizada.



Erro Comum: Construtor em Classe Abstrata

- Não é possível instanciar diretamente uma classe abstrata.
- Mesmo que ela tenha construtor, ele só pode ser chamado via `super()`.

Exemplo com erro de compilação:

Código

```
abstract class Animal {
    String nome;

    public Animal(String nome) {
        this.nome = nome;
    }
}

public class Teste {
    public static void main(String[] args) {
        Animal a = new Animal("Rex"); // ERRO
    }
}
```

Erro gerado:

- Animal is abstract; cannot be instantiated

Uso de super() na Classe Filha

- O construtor da classe abstrata é invocado pela classe filha usando super().
- Isso garante que os atributos herdados sejam corretamente inicializados.

Exemplo correto:

Código

```
abstract class Animal {  
    String nome;  
  
    public Animal(String nome) {  
        this.nome = nome;  
    }  
}  
  
class Cachorro extends Animal {  
  
    public Cachorro(String nome) {  
        super(nome); // chamada ao construtor da classe abstrata  
    }  
}
```

11. Exemplo de Código: Tudo Junto

```
1 abstract class Animal {
2     String nome;
3     Animal(String nome) { this.nome = nome; }
4
5     abstract void emitirSom();
6 }
7
8 class Cachorro extends Animal {
9     Cachorro(String nome) { super(nome); }
10
11     @Override
12     void emitirSom() {
13         System.out.println(nome + " diz: Au Au!");
14     }
15 }
```



12. Herança (extends)

- Uma classe herda de uma classe abstrata usando `extends`.
- Em Java, respeita-se a **herança simples**: uma classe só pode ter uma superclasse.
- A relação é do tipo **"é um"** (is-a): Cachorro ENTende-se como UM Animal.



Polimorfismo e Casting

13. Polimorfismo com Tipos Abstratos

- Podemos usar o tipo da classe abstrata como referência.
- Isso permite tratar diversos objetos concretos de forma genérica.

Flexibilidade

```
Animal a1 = new Cachorro("Rex");  
Animal a2 = new Gato("Mimi");  
a1.emitirSom(); // Au Au  
a2.emitirSom(); // Miau
```



Diferença: Tipos Abstratos vs Tipos Concretos

- Em Java, podemos declarar variáveis usando:
 - **Tipo abstrato** (mais genérico)
 - **Tipo concreto** (mais específico)
- **Tipo Abstrato:**
 - Referência usa a classe abstrata
 - Foca no **comportamento comum**
 - Permite maior **flexibilidade e polimorfismo**
- **Tipo Concreto:**
 - Referência usa a classe específica
 - Foca na **implementação detalhada**
 - Menor flexibilidade para substituição

Comparação

```
// Tipo abstrato (recomendado)
Animal a = new Cachorro("Rex");

// Tipo concreto
Cachorro c = new Cachorro("Rex");
```

- Use tipos abstratos para **desacoplar** o código
- Use tipos concretos quando precisar de **comportamentos específicos**

Diferença na Invocação de Métodos

- O tipo da referência determina **quais métodos podem ser chamados**.
- O tipo do objeto determina **qual implementação será executada**.

Exemplo

```
abstract class Animal {  
    public abstract void emitirSom();  
}  
  
class Cachorro extends Animal {  
    public void emitirSom() {  
        System.out.println("Au Au");  
    }  
  
    public void abanarRabo() {  
        System.out.println("Abanando o rabo");  
    }  
}
```

Uso

```
// Tipo abstrato  
Animal a = new Cachorro();  
a.emitirSom();    // OK  
// a.abanarRabo(); // ERRO de compilação
```

```
// Tipo concreto  
Cachorro c = new Cachorro();  
c.emitirSom();    // OK  
c.abanarRabo();   // OK
```

- **Polimorfismo:** o método chamado depende do objeto real
- **Restrição:** só é possível acessar métodos do tipo da referência

- Ao usar um **tipo abstrato**, você só acessa os métodos definidos na superclasse.
- Para acessar algo específico da subclasse, é necessário realizar o **casting**.

Casting em Ação

```
Animal a = new Cachorro();  
// a.abanarRabo(); // Erro de compilação  
((Cachorro) a).abanarRabo(); // OK: casting
```

- **Diferença importante:**
 - **Com tipo abstrato:** pode precisar de casting para acessar o específico.
 - **Com tipo concreto:** não precisa, pois a visibilidade é total.

- O compilador confia na indicação do programador durante o casting.
- Se o objeto real não for compatível, ocorrerá um erro fatal.

Risco: Erro em tempo de execução

```
Animal a = new Gato();
```

```
// ERRO: Gato não pode ser convertido para Cachorro  
((Cachorro) a).abanarRabo(); // ClassCastException
```

- Sempre valide o tipo com `instanceof` se houver dúvida sobre o objeto real.

- **Tipo Abstrato** → Generalização (visão genérica).
- **Tipo Concreto** → Especialização (visão detalhada).

Exemplo Clássico (Java Collections):

Boa Prática de Programação

```
// Use a abstração (List) - Generalização  
List<String> lista = new ArrayList<>();
```

- **Vantagem:** Evita prender o sistema ao ArrayList (concreto).
- Facilita a evolução para outras implementações de lista.

- O uso de tipos abstratos facilita a troca de implementações com impacto Zero.

Flexível (Desacoplado)

```
Animal a = new Cachorro();
```

```
// Posso mudar para gato  
// sem mexer no resto:  
Animal a = new Gato();
```

Rígido (Acoplado)

```
Cachorro c = new Cachorro();
```

```
// Se precisar mudar para  
// Gato, terei que alterar  
// todas as referências a 'c'.
```

SOLID e Boas Práticas

SOLID: O Problema do Acoplamento

- Quando dependemos de uma classe concreta, criamos um sistema rígido e difícil de evoluir.
- Exemplo: Uma classe Veterinario que só aceita Cachorro.

Código Acoplado (Ruim)

```
class Veterinario {  
    // Só funciona para cachorros!  
    public void examinar(Cachorro c) {  
        c.emitirSom();  
    }  
}
```

- **Problema:** Se precisarmos tratar um Gato, teremos que modificar a classe Veterinario.



- Ao depender da classe abstrata (`Animal`), o código torna-se flexível a novos tipos.

Código Desacoplado (Bom)

```
class Veterinario {  
    // Funciona para QUALQUER animal!  
    public void examinar(Animal a) {  
        a.emitirSom();  
    }  
}
```

- **Vantagem:** Podemos adicionar Gato, Papagaio ou Leão sem tocar em uma única linha da classe `Veterinario`.

- O princípio nos orienta a inverter a lógica de dependência:
- **Módulos de alto nível não devem depender de módulos de baixo nível (concreções).**
- Ambos devem depender de **abstrações**.

Frase Central da Engenharia

“Dependa de abstrações, não de classes concretas.”



- Métodos abstratos **exigem** ser implementados por uma subclasse.
- Se o método for `private`, a subclasse não terá visibilidade nem acesso para implementá-lo.

Erro de Compilação

```
abstract class Animal {  
    // ERRO: Modificadores incompatíveis  
    private abstract void emitirSom();  
}
```

- **Conclusão:** Métodos abstratos devem ser `public`, `protected` ou ter visibilidade de pacote (*default*).



- O modificador `final` impede a herança e a sobrescrita.
- O modificador `abstract` obriga a herança e a sobrescrita.
- São conceitos logicamente opostos.

Erro de Compilação

```
// ERRO: Uma classe não pode ser abstract e final
abstract final class Teste { }

abstract class Animal {
    // ERRO: Um método não pode ser abstract e final
    abstract final void emitirSom();
}
```

O que é static?

- **Atributo static:** pertence à classe e é compartilhado por todos os objetos.
 - **Método static:** pertence à classe e pode ser chamado sem criar um objeto.
 - Acesso direto usando o nome da classe.
-
- Classes abstratas podem conter métodos estáticos normalmente.
 - Como métodos static pertencem à classe (e não ao objeto), eles podem ser chamados mesmo que a classe não possa ser instanciada.

Exemplo: Métodos Utilitários

```
abstract class Conversor {  
    public static double celsiusParaFahrenheit(double c) {  
        return (c * 9/5) + 32;  
    }  
}
```

```
// Uso direto sem precisar de 'new':  
double f = Conversor.celsiusParaFahrenheit(25);
```



Permitido: Métodos final (Concretos)

- Você pode definir um método concreto como **final** em uma classe abstrata.
- Isso garante que a lógica daquele método seja **idêntica e imutável** para todas as subclasses.

Exemplo: Fluxo de Processamento Fixo

```
abstract class Relatorio {  
    // Cada subclasse decide o conteúdo  
    abstract void gerarCorpo();  
  
    // Nenhuma subclasse pode alterar o fechamento  
    public final void imprimirRodape() {  
        System.out.println("Relatorio gerado em: " + new java.util.Date());  
    }  
}
```



17. Quando usar Classe Abstrata?

- 1 Existe uma relação hierárquica forte.
- 2 Deseja-se evitar a duplicação de atributos e métodos comuns.
- 3 Deseja-se definir um "esqueleto" de algoritmo (Template Method) onde partes são fixas e partes variam.



18. Abstração em Cascata

- É possível uma classe abstrata herdar de outra classe abstrata.
- Nesse caso, a subclasse pode adicionar mais métodos abstratos ou implementar alguns dos herdados.

Exemplo

Animal (Abstrata) → Mamifero (Abstrata) → Humano (Concreta).



- **Evite Abstração Vazia:** Não crie uma classe abstrata se ela não tiver comportamento comum ou métodos a serem forçados.
- **Prefira Interfaces para Opcionais:** Se um comportamento não é comum a todos da hierarquia, coloque-o em uma interface.
- **Liskov Substitution:** Garanta que uma subclasse possa substituir sua superclasse sem quebrar o sistema.



Resumo

- Classes abstratas são ferramentas de design poderosas.
- Facilitam a manutenção ao centralizar lógica comum.
- Garantem que programadores sigam um modelo (contrato) ao criar novas subclasses.
- São o primeiro passo para um polimorfismo robusto e organizado.

